

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO5 ASSESSMENT TOOL 2 – Coding Challenge

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5 – Coding Challenge	Assessment:	Assessment Tool 2– Coding Challenge
Weightage:	40% of CIA	Total Marks:	100
CO5 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	S.Sirivally	Reg. No.:	192472371
Section / Batch:	2024- CSE(AI)	Date:	27-08-26

Code of Conduct

I S.Sirivally / 192472371 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sirivally.S

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

Q1. Write code to determine the optimal number of clusters (k) using both the Elbow Method and the Silhouette Score, computing the silhouette score from scratch (not `sklearn.metrics.silhouette_score`), and reconcile cases where the two methods disagree.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
df = pd.read_csv("data_file.csv")
print("Original Dataset:")
print(df)
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
X = df.select_dtypes(include=np.number)
print("\nNumeric Columns:")
print(X.columns.tolist())
if X.shape[1] == 0:
    print("\nERROR: No numeric columns found.")
    exit()
X = X.dropna()
print("\nDataset after removing missing values:")
print(X)
print("\nNumber of samples:", len(X))
```

```
n_samples = len(X)
if n_samples < 3:
    print("\nERROR: At least 3 samples are required.")
    exit()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# k must be smaller than the number of samples
max_k = min(8, n_samples - 1)
k_values = range(2, max_k + 1)
print("\nK values tested:")
print(list(k_values))
inertias = []
for k in k_values:
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    model.fit(X_scaled)
    inertias.append(model.inertia_)
print("\nElbow Method Results:")
for k, inertia in zip(k_values, inertias):
    print(
        "k =", k,
        "Inertia =", round(inertia, 4)
    )
```

```
plt.figure(figsize=(7, 5))
plt.plot(
    list(k_values),
    inertias,
    marker="o"
)
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.xticks(list(k_values))
plt.grid(True)
plt.show()

def calculate_silhouette(X, labels):
    n = len(X)
    silhouette_values = []
    unique_clusters = np.unique(labels)
    for i in range(n):
        current_cluster = labels[i]
        same_cluster_points = X[
            labels == current_cluster
        ]
        if len(same_cluster_points) <= 1:
            a = 0
        else:
            distances = np.linalg.norm(
                same_cluster_points - X[i],
```

```

        axis=1
    )
    # Remove distance from point to itself
    distances = distances[distances != 0]
    if len(distances) == 0:
        a = 0
    else:
        a = np.mean(distances)
    average_distances = []
    for cluster in unique_clusters:
        if cluster == current_cluster:
            continue
        other_cluster_points = X[
            labels == cluster
        ]
        distances = np.linalg.norm(
            other_cluster_points - X[i],
            axis=1
        )
        average_distance = np.mean(distances)
        average_distances.append(
            average_distance
        )
    if len(average_distances) == 0:
        b = 0
    else:

```

```

        b = min(average_distances)
    denominator = max(a, b)
    if denominator == 0:
        s = 0
    else:
        s = (b - a) / denominator
    silhouette_values.append(s)
return np.mean(silhouette_values)

silhouette_scores = []
print("\nSilhouette Score Results:")
for k in k_values:
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    labels = model.fit_predict(X_scaled)
    score = calculate_silhouette(
        X_scaled,
        labels
    )
    silhouette_scores.append(score)
print(
    "k =", k,
    "Silhouette Score =",
    round(score, 4)

```

```
)  
plt.figure(figsize=(7, 5))  
plt.plot(  
    list(k_values),  
    silhouette_scores,  
    marker="o"  
)  
plt.xlabel("Number of Clusters (k)")  
plt.ylabel("Silhouette Score")  
plt.title("Silhouette Score")  
plt.xticks(list(k_values))  
plt.grid(True)  
plt.show()  
best_k_silhouette = list(k_values)[  
    np.argmax(silhouette_scores)  
]  
best_silhouette = max(silhouette_scores)  
print("\n=====")  
print("BEST SILHOUETTE RESULT")  
print("=====")  
print(  
    "Best k =",  
    best_k_silhouette  
)  
print(  
    "Silhouette Score =",
```

```

        round(best_silhouette, 4)
    )
    x = np.array(list(k_values))
    y = np.array(inertias)
    x1 = x[0]
    y1 = y[0]
    x2 = x[-1]
    y2 = y[-1]
    distances = []
    for i in range(len(x)):
        numerator = abs(
            (y2 - y1) * x[i]
            - (x2 - x1) * y[i]
            + x2 * y1
            - y2 * x1
        )
        denominator = np.sqrt(
            (y2 - y1) ** 2
            + (x2 - x1) ** 2
        )
        distance = numerator / denominator
        distances.append(distance)
    best_k_elbow = x[
        np.argmax(distances)
    ]
    print("\n=====")

```



```
print("ELBOW METHOD RESULT")
print("=====")
print(
    "Estimated Elbow k =",
    best_k_elbow
)
print("\n=====")
print("FINAL RECOMMENDATION")
print("=====")
if best_k_elbow == best_k_silhouette:
    final_k = best_k_elbow
    print(
        "Both methods agree."
    )
    print(
        "Recommended number of clusters =",
        final_k
    )
else:
    print(
        "The Elbow and Silhouette methods disagree."
    )
    print(
        "Elbow k =",
        best_k_elbow
    )
```

```

print(
    "Silhouette k =",
    best_k_silhouette
)
print(
    "Silhouette score provides a clearer numerical"
    " measure of cluster separation."
)
final_k = best_k_silhouette
print(
    "Recommended k based on higher silhouette quality =",
    final_k
)
final_model = KMeans(
    n_clusters=int(final_k),
    random_state=42,
    n_init=10
)
final_labels = final_model.fit_predict(X_scaled)
result = X.copy()
result["Cluster"] = final_labels
print("\n=====")
print("FINAL CLUSTER ASSIGNMENTS")
print("=====")

print(result)

```



